

INTERNATIONAL POSTCODE SYSTEM (IPCS)

автор

Бартош Цеслицки

Рочдейл, 2018

Допущения системы

Основная идея системы — создать единую систему почтовых индексов для всей Земли.
Главная цель:

КОДИРОВАНИЕ ГЕОГРАФИЧЕСКИХ КООРДИНАТ КАЖДОГО ПОЧТОВОГО ИНДЕКСА ПРЯМО В САМОЙ ЗАПИСИ.

Такая система устранил необходимость в огромной глобальной базе данных с местоположением каждого почтового индекса. Она также снизит или полностью устранил затраты на геокодирование, а также значительно ускорит компьютерные системы, ищущие географическое положение для запрашиваемого адреса.

Допущения:

- Площадь, обозначаемая почтовым индексом, не должна превышать 1000 м², чтобы можно было легко и точно добраться до нужного места с помощью карты или спутниковой навигации.
- Система должна охватывать каждую часть планеты независимо от населения, плотности застройки, рельефа или типа поверхности.
- Почтовые индексы должны вычисляться / создаваться на основе алгоритма, а не присваиваться произвольно.
- Географические координаты можно вычислить из каждой записи почтового индекса, обозначающей центр участка.

International Postcode System (IPCS) должен:

- опираться на латинский алфавит как наиболее распространённый в мире.
- иметь относительно небольшое число символов, чтобы его легко было запомнить.
- не содержать специальных символов, характерных для какой-то одной страны.
- исключать наиболее частые ошибки при написании и чтении (то есть путаницу буквы O с цифрой 0 и буквы I с цифрой 1).

Построение секторов

Для вычисления International Postcode System (IPCS) мы используем широту и долготу в десятичных градусах. Долгота находится в диапазоне $(-180, 180>$, а широта — в диапазоне $<-90, 90>$. Первый, очень важный шаг вычисления международного почтового индекса — округление широты и долготы до четверти от 0,001.

Пример:

Предположим, что число $t \in <0.000, 0.001>$, а число x — наше округление. Если:

$$x \in <0.000, 0.000125) \rightarrow x = 0.000$$

$$x \in <0.000125, 0.000375) \rightarrow x = 0.00025$$

$$x \in <0.000375, 0.000625) \rightarrow x = 0.0005$$

$$x \in <0.000625, 0.000875) \rightarrow x = 0.00075$$

$$x \in <0.000875, 0.001> \rightarrow x = 0.001$$

Благодаря этим округлениям, вся Земля делится на зоны. Полученные координаты идентифицируют каждый созданный сектор на Земле и одновременно являются его центром. Это выполняет допущение, сделанное в начале.

Фрагмент сетки выглядит следующим образом:

Структура секторов, основанная на координатах, приводит к тому, что площадь каждого сектора, образованного округлением широты и долготы, уменьшается по мере приближения к полюсам Земли. Наибольшие площади будут, соответственно, у экватора и составят около 728 м², что отвечает нашему допущению.

Мы можем вычислить количество таким образом созданных секторов. Числа округляются до четверти тысячной, поэтому каждая тысячная делится на 4 части.

Для долготы у нас есть числовой диапазон $(-180, 180>$. Через x обозначим количество делений. Оно равно:

$$x = 360 * 1000 * 4 \quad (1)$$

Для широты у нас есть числовой диапазон $<-90, 90>$. Через y обозначим количество делений. Оно равно:

$$y = 181 * 1000 * 4 \quad (2)$$

(Значение 181 вместо 180 учитывает то, что диапазон широты $<-90, 90>$ замкнут с обеих сторон — южный полюс (-90°) и северный полюс (90°) являются двумя разными точками — в отличие от диапазона долготы $(-180, 180>$, замкнутого лишь с одной стороны, поскольку -180° и 180° обозначают один и тот же меридиан.)

Используя (1) и (2), вычисляем количество секторов для всей Земли:

$$\text{Number of sectors} = x * y = (360 * 1000 * 4) * (181 * 1000 * 4) = 1\,042\,560\,000\,000 \quad (3)$$

Остаётся лишь построить Международный Почтовый Индекс (IPCS) для каждого из полученных/созданных секторов.

Построение International Postcode System (IPCS)

Используя латинские буквы и цифры, мы располагаем 10 цифрами и 26 буквами. Мы убираем из алфавита буквы О и I (чтобы соответствовать допущениям системы). Таким образом получаем следующие 34 символа: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, J, K, L, M, N, P, Q, R, S, T, U, V, W, X, Y, Z}. На основе этих 34 символов создаём числовую систему — 34-ричную систему счисления. Слева — числа в 34-ричной системе, справа — их эквивалент в десятичной системе.

0 = 0	8 = 8	G = 16	Q = 24	Y = 32
1 = 1	9 = 9	H = 17	R = 25	Z = 33
2 = 2	A = 10	J = 18	S = 26	
3 = 3	B = 11	K = 19	T = 27	
4 = 4	C = 12	L = 20	U = 28	
5 = 5	D = 13	M = 21	V = 29	
6 = 6	E = 14	N = 22	W = 30	
7 = 7	F = 15	P = 23	X = 31	

Каждое число в 34-ричной системе можно представить в десятичной системе следующим образом:

$$\dots dcba_{34} = f(a) \times 34^0 + f(b) \times 34^1 + f(c) \times 34^2 + f(d) \times 34^3 + \dots$$

, где f — функция, сопоставляющая каждой цифре 34-ричной системы её эквивалент в десятичной системе (см. (4)). Здесь a, b, c, d — цифры 34-ричной системы.

Пример:

$$\begin{aligned} AGZ15Z_{34} &= f(Z) \times 34^0 + f(5) \times 34^1 + f(1) \times 34^2 + f(Z) \times 34^3 + f(G) \times 34^4 + f(A) \times 34^5 \\ &= 33 \times 34^0 + 5 \times 34^1 + 1 \times 34^2 + 33 \times 34^3 + 16 \times 34^4 + 10 \times 34^5 \\ &= 33 \times 1 + 5 \times 34 + 1 \times 1156 + 33 \times 39304 + 16 \times 1336336 + 10 \times 45435424 \\ &= 33 + 170 + 1156 + 1297032 + 21381376 + 454354240 = 477034007 \end{aligned}$$

Как видно, запись в 34-ричной системе гораздо короче. Нам потребуется 8 символов для генерации почтовых индексов для всей планеты. Максимальное число символов в действующей английской и ирландской системах почтовых индексов равно 7, в Японии — 7 цифр, в Нидерландах — 6 символов и т.д. Учитывая длину почтовых индексов в других странах, а также то, что International Postcode System — глобальная система, мы предполагаем, что 8 символов приемлемы и соответствуют допущениям нашей системы.

Наибольшее число, которое мы можем сохранить, используя 8 цифр в нашей 34-ричной системе, равно (подробные вычисления опущены):

$$ZZZZZZZZ_{34} = 1\,785\,793\,904\,895$$

Отсюда следует, что у Земли может быть 1 785 793 904 896 почтовых индексов (это число на единицу больше максимального, потому что 00000000 тоже может быть кодом). Из предыдущего вычисления (3) мы знаем количество секторов в нашей структуре — оно равно 1 042 560 000 000. Это число меньше 1 785 793 904 895, поэтому 8 цифр в 34-ричной системе

достаточно для однозначного определения каждого сектора. Площади секторов из-за этой структуры не равны между собой. Наибольшие из них расположены вблизи экватора и составляют $\approx 728 \text{ м}^2$. Площадь Земли составляет около $510\,100\,000 \text{ км}^2 = 510\,100\,000\,000\,000 \text{ м}^2$. Чтобы вычислить среднюю площадь сектора, выполним простую операцию:

$$510\,100\,000\,000\,000 \text{ м}^2 / 1\,042\,560\,000\,000 \approx 489.2764 \text{ м}^2$$

Таким образом, $ZZZZZZZZ_{34}$ соответствует нашим допущениям, а площадь каждого сектора меньше 1000 м^2 .

Теперь остаётся преобразовать центр сектора в формат International Postcode System (IPCS).

Для наших целей мы представляем целую часть отрицательных чисел следующим образом:

- Для долготы отрицательная целая часть x преобразуется в $360 + x$.
- Для широты отрицательное целое y преобразуется в $180 + y$.

Дробная часть остаётся без изменений. Итого:

$$\text{Longitude } -XXX.xxxxx \rightarrow (360 + (-XXX)).xxxxx$$

$$\text{Latitude } -YYY.yyyyy \rightarrow (180 + (-YYY)).yyyyy$$

Примеры преобразования:

- Число -75.43725 для долготы становится 285.43725 , потому что $360 + (-75) = 285$.
- Число -0.411250 для долготы становится 360.411250 , потому что $360 + 0 = 360$.
- Число -15.44375 для широты становится 165.44375 , потому что $180 + (-15) = 165$.

При создании алгоритма следует учитывать, что в этой записи южный полюс на 90 градусах широты и северный полюс на -90 градусах широты считаются одинаковыми.

С учётом преобразования отрицательных чисел и округления общий вид координат, записанных в десятичной системе, таков:

$$\text{Longitude: } XXX.xxxqq, \text{ where } XXX \in \{x \in \mathbb{N}: 000 \leq x \leq 360\}, xxx \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}, qq \in \{00, 25, 50, 75\}$$

$$\text{Latitude: } YYY.yyygg, \text{ where } YYY \in \{x \in \mathbb{N}: 000 \leq x \leq 180\}, yyy \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}, gg \in \{00, 25, 50, 75\}$$

На следующем шаге каждый элемент чисел $XXX.xxxqq$ и $YYY.yyygg$ преобразуется в двоичную систему.

$$XXX_{10} = XXXXXXXXX_2 \text{ — max. } 360, \text{ requires } 9 \text{ bits.}$$

$$xxx_{10} = xxxxxxxxxxx_2 \text{ — max. } 999, \text{ requires } 10 \text{ bits.}$$

$$qq_{10} = qq_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

$$YYY_{10} = YYYYYYYY_2 \text{ — max. } 180, \text{ requires } 8 \text{ bits.}$$

$$yyy_{10} = yyyyyyyyyy_2 \text{ — max. } 999, \text{ requires } 10 \text{ bits.}$$

$$gg_{10} = gg_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

Затем располагаем двоичные числа в следующем порядке:

$$XXXXXXXXXX YYYYYYYY xxxxxxxxxxxx yyyyyyyyyy qq gg_2$$

Берём число не менее 41 бита, обнуляем его и записываем в него вышеуказанные биты справа налево, сохраняя их порядок. Полученное число равно:

$$XXXXXXXXXX YYYYYYYY xxxxxxxxxxxx yyyyyyyyyy qq gg_2$$

На последнем шаге мы преобразуем двоичное число в 34 -ричную систему. Полученное 34 -ричное число и есть наш Международный Почтовый Индекс (IPCS). Координаты центра

сектора закодированы в нашей записи Международного Почтового Индекса (IPCS).

Порядок, в котором записаны биты, очень важен, потому что при округлении и представлении согласно нашей конвенции координата с наибольшим числовым значением — это 360.99975, 180.99975. Преобразуя её в двоичную систему, получаем число:

$$10110100010110100111110011111110011111112 = 1\,549\,224\,509\,055_{10}$$

Полученное число меньше максимального числа 34-ричного кода, которое равно $ZZZZZZZZ_{34} = 1\,785\,793\,904\,895_{10}$. Изменение порядка битов могло бы превысить это максимальное значение, что сделало бы невозможным создание почтовых индексов для всех секторов.

Чтобы получить координаты центра сектора (Международный Почтовый Индекс), мы выполняем операцию в обратном порядке.

Примеры:

Международный Почтовый Индекс (IPCS) для координат -1.347120, 53.983489 мы конструируем следующим образом:

Округляем, преобразуем координаты и получаем 359.34700, 53.98350

Преобразуем каждый элемент координат, то есть числа 359, 347, 00, 53, 983, 50, в двоичную систему, как описано выше, и получаем: 101100111_2 , 0101011011_2 , 00_2 , 01101010_2 , 1111010111_2 , 10_2

Располагаем числа в правильном порядке: 101100111_2 , 0011010_{21} , 0101011011_2 , 1111010111_2 , 00_2 , 10_2

Создаём число $1011001110011010101010110111110101110010_2$

Преобразуем двоичное число в 34-ричную систему и получаем $VCPM6TKY_{34}$

Полученное число — это Международный Почтовый Индекс (IPCS): VCPM 6TKY

Координаты для Международного Почтового Индекса 1GL8 YT7F вычисляются следующим образом:

Преобразуем 34-ричное число $1GL8YT7F_{34}$ в двоичную систему и получаем $000010010001100101100000011010011011010012$

Извлекаем числа 00001001_2 , 0011001_{20} , 1100000011_2 , 0100110110_2 , 10_2 , 01_2

Меняем порядок на 00001001_2 1100000011_2 , 10_2 , 0011001_{20} 0100110110_2 , 01_2

Преобразуем двоичные числа в десятичные: 18, 771, 50, 50, 310, 25

Из полученных чисел получаем координаты 18.77150, 50.31025

Функция для генерации и вычисления координат из Международного Почтового Индекса

Ниже приведены две функции, написанные на C#. Первая строит Международный Почтовый Индекс, вторая — вычисляет координаты из Международного Почтового Индекса.

```
public string GeneratePostcode(double Lon, double Lat)
{
    if ((Lon > 180) || (Lon < -180) || (Lat > 90) || (Lat < -90)) return "";
    if (Lat > 89.99975) Lat = 89.99975;
    if (Lat < -89.99975) Lat = -89.99975;

    string[] Coder = { "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E", "F", "G", "H", "J",
        "K", "L", "M", "N", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z" };

    if (Lat < 0)
    {
        Lat = ((long)Lat + 180) + Math.Abs(Lat - (long)Lat);
    }
    if (Lon < 0)
    {
        Lon = ((long)Lon + 360) + Math.Abs(Lon - (long)Lon);
    }

    string PartPostcodeLat = Lat.ToString("000.000000");
    string PartPostcodeLon = Lon.ToString("000.000000");

    try
    {
        string[] StrsLat = PartPostcodeLat.Split('.');
        string[] StrsLon = PartPostcodeLon.Split('.');

        ulong OptionsLat = 5;
        long Precision = Convert.ToInt32(StrsLat[1].Substring(3));
        if (Precision < 125)
        {
            OptionsLat = 0;
        }
        else if (Precision < 375)
        {
            OptionsLat = 1;
        }
        else if (Precision < 625)
        {
            OptionsLat = 2;
        }
        else if (Precision < 875)
        {
            OptionsLat = 3;
        }

        ulong TempHiLat = Convert.ToUInt64(StrsLat[0]);
        TempHiLat = TempHiLat << 24;
        ulong TempLoLat = 0;
        if (StrsLat.Length > 1)
        {
            TempLoLat = Convert.ToUInt64(StrsLat[1].Substring(0, 3));
            if (OptionsLat == 5)
            {
                TempLoLat++;
            }
            else
            {
                TempLoLat++;
            }
        }
    }
}
```

```

        TempHiLat = TempHiLat | OptionsLat;
    }

    TempLoLat = TempLoLat << 4;
}

ulong OptionsLon = 5;
Precision = Convert.ToInt32(StrsLon[1].Substring(3));
if (Precision < 125)
{
    OptionsLon = 0;
}
else if (Precision < 375)
{
    OptionsLon = 1;
}
else if (Precision < 625)
{
    OptionsLon = 2;
}
else if (Precision < 875)
{
    OptionsLon = 3;
}

ulong TempHiLon = Convert.ToUInt64(StrsLon[0]);
TempHiLon = TempHiLon << 32;
ulong TempLoLo = 0;
if (StrsLon.Length > 1)
{
    TempLoLo = Convert.ToUInt64(StrsLon[1].Substring(0, 3));
    if (OptionsLon == 5)
    {
        TempLoLo++;
    }
    else
    {
        TempHiLon = TempHiLon | (OptionsLon * 4);
    }
}

TempLoLo = TempLoLo << 14;
}

ulong UlongResHi = TempHiLat | TempHiLon | TempLoLat | TempLoLo;

ulong Pos = 0;
string Res = "";
for (int i = 7; i >= 0; i--)
{
    Pos = UlongResHi / (ulong)Math.Pow(34, i);
    UlongResHi -= (Pos * (ulong)Math.Pow(34, i));
    Res += Coder[Pos];
}

return Res;
}
catch
{
    return "";
}
}

public bool ReversePostcode(string Postcode, out double Lat, out double Lon)
{
    if (Postcode.Length == 8)
    {

```

```

Postcode = Postcode.ToUpper();
string Coder = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
ulong Hi = 0;

for (int i = 0; i < 8; i++)
{
    Hi += (ulong)Coder.IndexOf(Postcode[i]) * (ulong)Math.Pow(34, (7 - i));
}

ulong OptionLat = (Hi & 3);
double HalfLat = 0;
switch (OptionLat)
{
    case 1:
        HalfLat = 0.00025;
        break;
    case 2:
        HalfLat = 0.0005;
        break;
    case 3:
        HalfLat = 0.00075;
        break;
}

ulong OptionLon = (Hi & 12) >> 2;
double HalfLon = 0;
switch (OptionLon)
{
    case 1:
        HalfLon = 0.00025;
        break;
    case 2:
        HalfLon = 0.0005;
        break;
    case 3:
        HalfLon = 0.00075;
        break;
}

long LoLat = ((long)Hi >> 4) & 1023;
long HiLat = ((long)Hi >> 24) & 255;
if (HiLat > 180)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLat > 90)
{
    HiLat = HiLat - 180;
    Lat = HiLat - ((LoLat / 1000.0) + HalfLat);
}
else
{
    Lat = HiLat + ((LoLat / 1000.0) + HalfLat);
}

long LoLon = ((long)Hi >> 14) & 1023;
long HiLon = ((long)Hi >> 32);
if (HiLon > 360)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLon > 180)

```

```
{
    HiLon = HiLon - 360;
    Lon = HiLon - ((LoLon / 1000.0) + HalfLon);
}
else
{
    Lon = HiLon + ((LoLon / 1000.0) + HalfLon);
}

return true;
}
else
{
    Lat = 0;
    Lon = 0;
    return false;
}
}
```